LOGICA PARA CIENCIAS DE LA COMPUTACION

(DCIC-5704)

INFORME

Nonograma

Comisión 54

Integrante: Nicolas Bernardini LU: 140978.

Fecha de entrega: 03/05/2024

Indice

Introducción 3
Requerimientos funcionales 4
    Inicializar estado de las pistas 4
    Calcular FilaSat y ColSat 5
    Actualizar pistas al hacer una jugada 6
    Determinar si el jugador gano 7
  Funcionalidades extra 8
  Casos de prueba utilizados 9
    updateCluesSatisfactionLists 9
    checkClue 10
    insertCluesatInClueSatisfactionList 11
    CheckWin 12
Aspectos positivos de la resolución 12
Problemas 13

Introducción

Este informe presenta la documentación sobre la implementación en Prolog del juego Nonograma. Se proporciona una descripción detallada de las estrategias especificas del lenguaje Prolog empleadas para abordar los requerimientos funcionales, así como una selección de casos de prueba significativos que ilustran la efectividad y precisión del sistema desarrollado.

Requerimientos funcionales

Inicializar estado de las pistas

Al comenzar el juego, es posible que el tablero ya satisfaga algunas o todas las pistas. Para gestionar esto, la función **updateCluesSatisfactionLists**/5 se encarga de inicializar el estado de satisfacción de las pistas para las filas y columnas del tablero del juego.

La representación del estado de satisfacción se realiza mediante dos listas: una para las filas y otra para las columnas. En estas listas, cada elemento indica si la fila o columna correspondiente satisface sus pistas o no. Se utiliza el valor booleano true para indicar que la fila o columna cumple con sus pistas y false en caso contrario.

Como se dijo anteriormente el estado inicial de estas listas se obtienen con el predicado **updateCluesSatisfactionLists**(Grilla, PistasFilas, PistasColumns, Estado de filas, Estado de columnas).

Donde:

- Estado de filas y Estado de columnas son las listas previamente mencionadas que contienen el estado de todas las filas y columnas del tablero.

El algoritmo que sigue **updateCluesSatisfactionLists** es el siguiente:

1. Se calcula el tamaño de PistasFilas y PistasColumns.
2. Se crea una lista del mismo tamaño de PistasFilas y otra de tamaño de PistasColumns.
3. Por cada fila y columna
    - Se verifica el estado de satisfacción.
    - Con el resultado obtenido, se actualiza la lista de satisfacción en su posición correspondiente.

Para determinar si una fila o columna cumple con sus pistas respectivas, se emplea el predicado **checkClue/3**. Una vez que se ha evaluado el cumplimiento de las pistas para una fila o columna específica, se utiliza el predicado **insertCluesatInClueSatisfactionList/4** para actualizar el estado de satisfacción en la lista correspondiente. Se proporcionarán detalles completos sobre su implementación en secciones posteriores del informe.

Calcular FilaSat y ColSat

Al realizar un movimiento (colocar una “X” o “#” en el tablero) mediante el predicado **put/8**. Es necesario calcular FilaSat y ColSat para garantizar la coherencia de juego. Para lograr esto, se emplea el predicado **checkClue/3**, que sigue la siguiente descripción:

checkClue(CellList, Clues, CluesMet).

Donde:

- CellList es una lista que representa una fila o columna especifica de la grilla.
- Clues es la lista de pistas específicas de CellList.
- CluesMet es 1 si CellList satisface las pistas asociadas de Clues, y 0 en caso contrario.

checkClue llama internamente a **checkClueAux/4** que implementa el algoritmo para verificar la satisfacción de las pistas en detalle.

checkClueAux(CellList, Clues, FilledCell, CluesMet)

- FilledCell es un contador que representa la cantidad de celdas pintadas adyacentes.

El algoritmo en Prolog de checkClueAux es el siguiente:

1. Si no hay más celdas por mirar y quedan pistas sin completar CluesMet retorna 0
2. Si no hay más pistas por mirar (o sea se satisficieron todas)
    - Si lo que queda de CellList no hay cuadrados pintados, CluesMet retorna 1, si no retorna 0.
3. Si es la última celda
    - Si esta vacía y se cumple la ultima regla CluesMet retorna 1 si no retorna 0.
    - Si esta pintada y se cumple la ultima regla CluesMet retorna 1, si no retorna 0.
4. Si CellList tiene más de 1 celda
    - Si la primera celda es vacía
        - Si No hay cuadrados pintados adyacentes se sigue a la siguiente celda
        - Si hay cuadrados pintados adyacentes
            - Si no se satisface la primera regla retorna 0
            - Si satisface la primera regla se sigue con el resto de la lista de celdas y con la siguiente pista
5. Si la primera celda no es vacía
    - Se incrementa el contador de FilledCell y se sigue revisando el resto de FilledCell con las mismas pistas.

Actualizar pistas al hacer una jugada

Al hacer una jugada (colocar una X o pintar un cuadrado) se deben actualizar las pistas de la fila y columna afectadas, para esto en Game.js se utiliza el predicado **put/8** detallado en el enunciado del proyecto, este retorna RowSat y ColSat que es utilizado para modificar el estado de satisfacción de la fila y columna afectado mediante el predicado **insertCluesatInClueSatisfactionList/4**. Su definición:

insertCluesatInClueSatisfactionList(EstadoDePistas, posición, Cluesat, NuevoEstadoDePistas).

Donde:

- EstadoDePistas es la lista de satisfacción de pistas en la fila o columna.
- Posición es la fila o columna que fue modificada.
- ClueSat es 1 o 0 dependiendo si se satisface las pistas.
- NuevoEstadoDePistas es la lista resultante de actualizar la lista EstadoDePistas en la posición “Posición” con el valor de ClueSat.

La estrategia utilizada es la siguiente:

1. Si posición = 0 (caso base)
    - Si ClueSat = 1, se devuelve la lista EstadoDePistas remplazando el valor de en la posición 0 por true.
    - Si ClueSat = 0, se devuelve la lista EstadoDePistas remplazando el valor de en la posición 0 por false.
2. Si posición > 0 (caso general)
    - Se busca recursivamente la posición en la lista EstadoDePistas que corresponde a la fila o columna modificada. Retornando la lista modificada, donde el valor en la posición correspondiente se actualiza según el valor de ClueSat.

Determinar si el jugador gano

Después de realizar un movimiento en el juego, se verifica si el jugador ha ganado utilizando el predicado **playerWin/3**. Este tiene la siguiente forma:

PlayerWin(EstadoDeFilas, EstadoDeColumnas, ElJugadorGano).

Donde:

- ElJugadorGano retorna true si EstadoDeFilas y EstadoDeColumnas únicamente contienen true, retorna false en caso contrario.

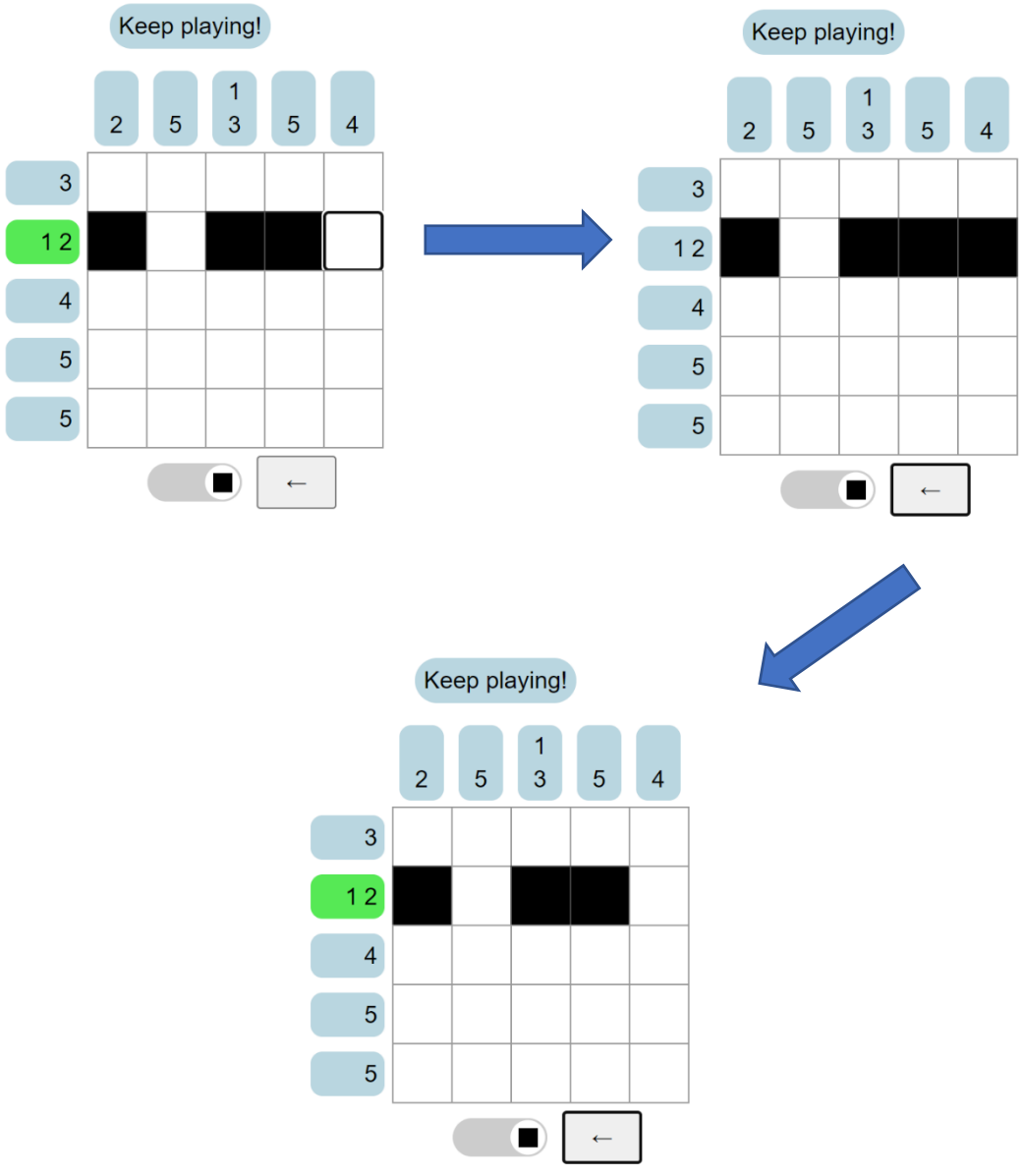
PlayerWin se implementó con la siguiente lógica:

1. Se recorre EstadoDeFilas:
    - Si se encuentra un false retorna false mediante ElJugadorGano.
2. Si no se encuentra false, se recorre EstadoDeColumnas
    - Si se encuentra false retornar false mediante ElJugadorGano, si no retornar true.

Funcionalidades extra

Se ha añadido una funcionalidad adicional al juego que permite al jugador retroceder los movimientos realizados. Esta característica proporciona una forma conveniente de corregir errores o reconsiderar decisiones previas durante el juego.

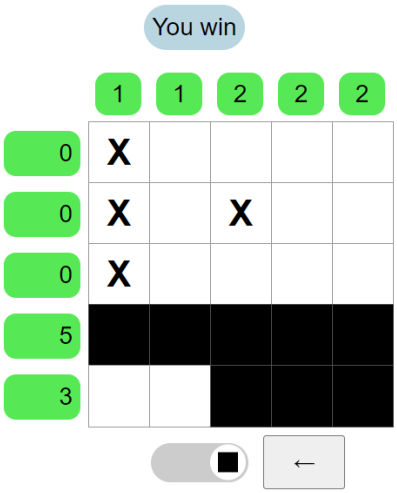
En la implementación en JavaScript, se ha creado un arreglo que actúa como un historial de tablero. Este arreglo guarda el estado del tablero cada vez que se realiza un movimiento. Cuando el jugador desea retroceder un movimiento, se utiliza este historial para restaurar el estado anterior del tablero. Para calcular el estado de satisfacción correspondiente a la nueva configuración del tablero después de retroceder un movimiento, se reutiliza el predicado **updateCluesSatisfactionLists/5.**



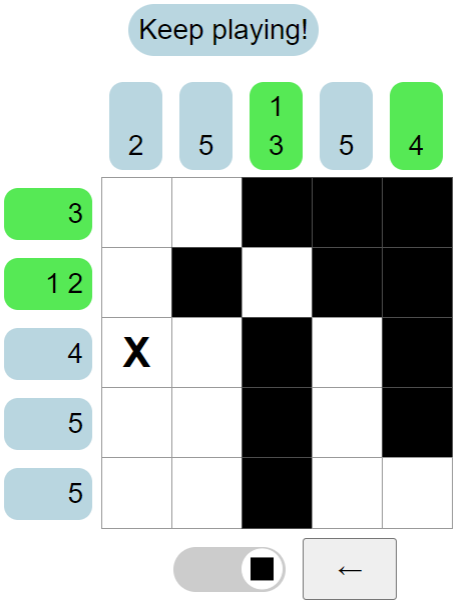
Casos de prueba utilizados

updateCluesSatisfactionLists

A continuación, se presentan dos casos de prueba representativos que ilustran su comportamiento:



Este caso de prueba simula la inicialización del juego con un tablero donde todas las pistas se satisfacen desde el principio.

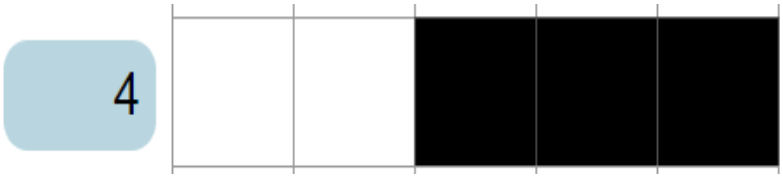


En este caso de prueba, se establece un escenario donde algunas pistas se satisfacen inicialmente, mientras que otras no.

Dado que el método de **updateCluesSatisfactionLists** no es excesivamente complicado y estos dos casos de prueba cubren escenarios representativos, se ha decidido no incluir todos los casos utilizados para evitar saturar la documentación. Estos casos seleccionados proporcionan una muestra adecuada del funcionamiento del predicado y aseguran su robustez y precisión en el juego.

checkClue

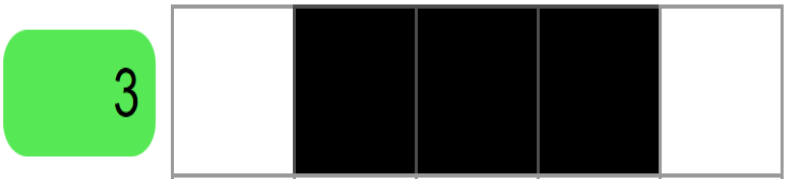
El predicado checkClue es una parte fundamental del juego y ha sido sometido a una serie de casos de prueba exhaustivos para garantizar su correcto funcionamiento en diversas situaciones. Se han representado todos los casos como filas para una mejor comprensión, aunque en términos prácticos, el predicado funciona de manera equivalente con las columnas. A continuación, se presentan algunos ejemplos representativos de estos casos de prueba:



Este caso verifica el comportamiento del predicado cuando no quedan más celdas por revisar y las pistas no se satisfacen.



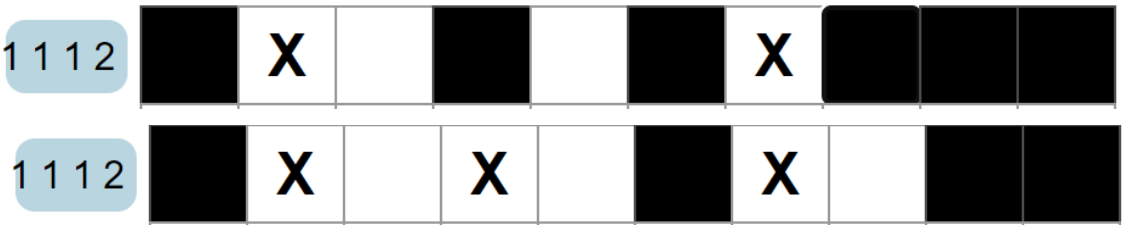
En este caso de prueba comprueba si el predicado reconoce correctamente una fila completa satisface todas las pistas.



Aquí se verifica si el predicado detecta adecuadamente una pista que se cumple en el medio de una fila.



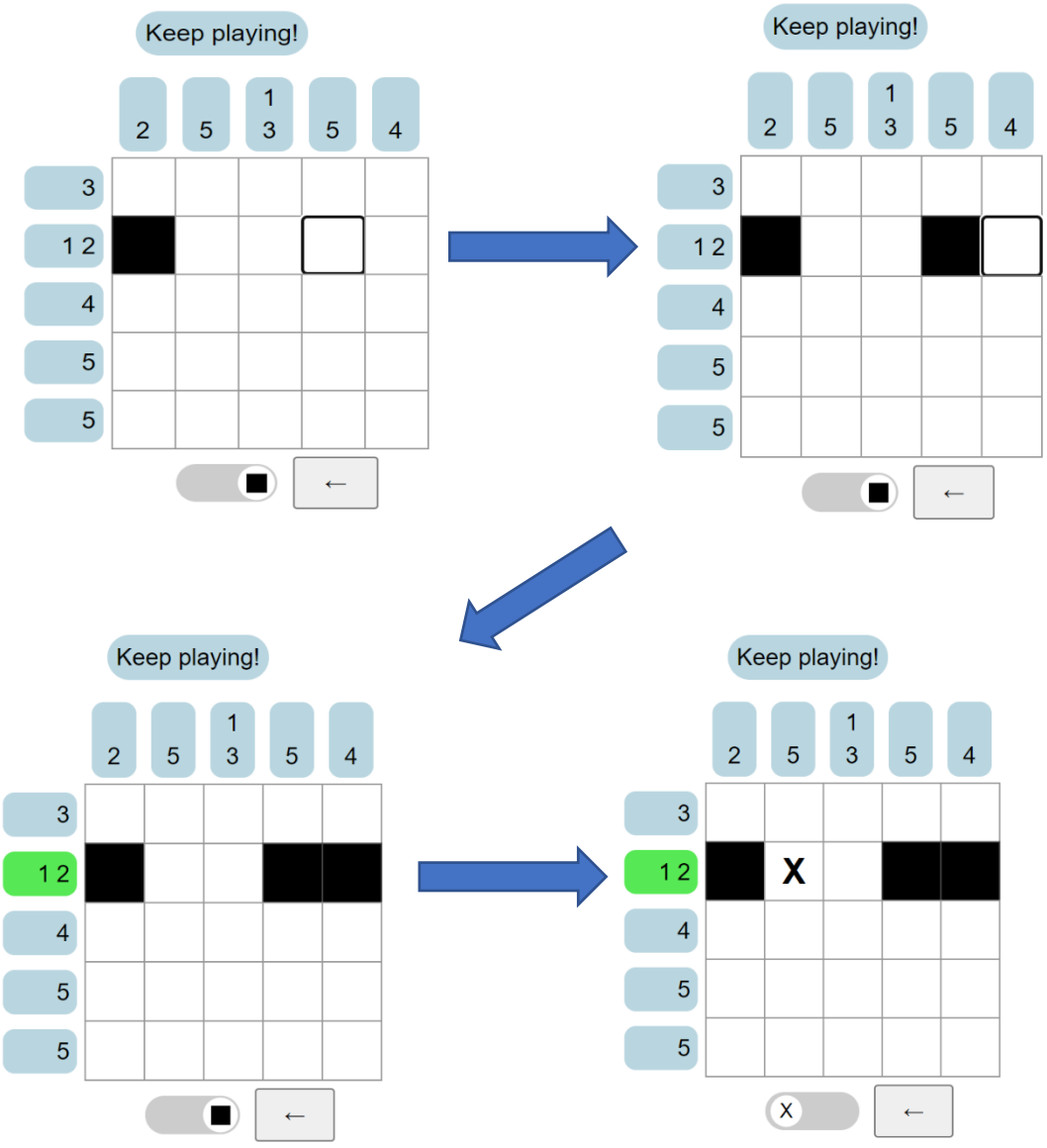
En este caso se testean varias pistas que tienen celdas marcadas intercaladas con celdas vacías.



Estos casos verifican que un cambio en una celda pintada hace que las pistas ya no se satisfagan.

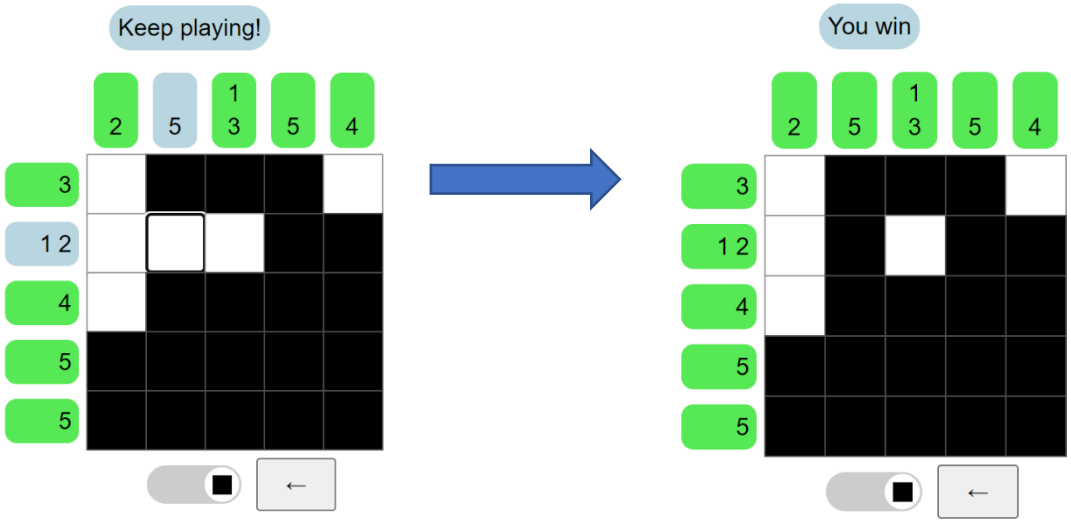
insertCluesatInClueSatisfactionList

El predicado es una parte esencial del proceso de actualización del estado de satisfacción de las pistas del juego. Se utiliza en predicado **updateCluesSatisfactionLists** y luego de ejecutar el predicado **put**, con el fin de actualizar las listas que representan el estado de satisfacción de las pistas. Dado que este predicado no es muy complejo, se presenta un único caso de prueba que abarca su uso general:



CheckWin

Dada la poca complejidad del algoritmo se muestra un único caso de prueba que contempla un escenario completo donde se verifica tanto las pistas de una fila como de una columna al mismo tiempo:



Aspectos positivos de la resolución

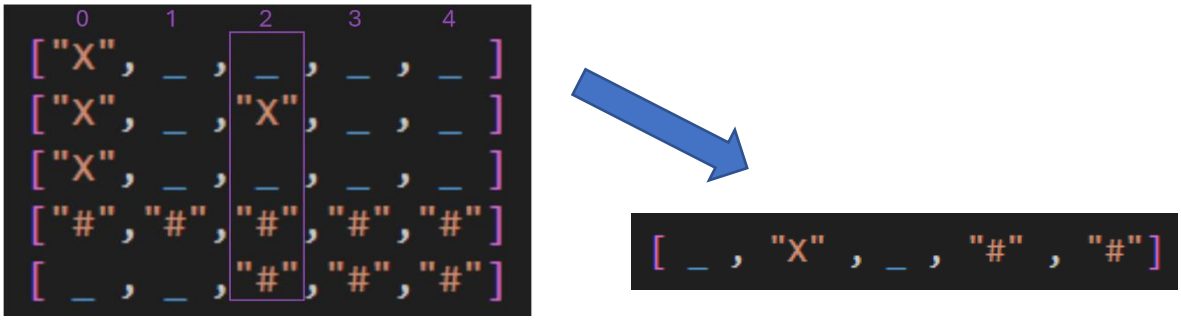
El código implementado para el juego destaca por su legibilidad y eficiencia en el tiempo de ejecución, abordando de manera efectiva los diversos casos posibles que pueden surgir durante el juego. Algunos aspectos positivos del diseño y la implementación son:

Legibilidad y Eficiencia: El código se ha desarrollado de manera clara y legible, lo que facilita su comprensión y mantenimiento. Además, se ha optimizado para garantizar un tiempo de ejecución decente incluso en escenarios con gran cantidad de pistas y una grilla grande.

Escalabilidad: Los predicados **checkClue/3** y **updateCluesSatisfactionLists/5** han sido diseñados para ser escalables, lo que significa que pueden manejar tableros de cualquier tamaño y complejidad sin comprometer su funcionalidad. Esta característica garantiza que el juego sea jugable independientemente de las dimensiones del tablero.

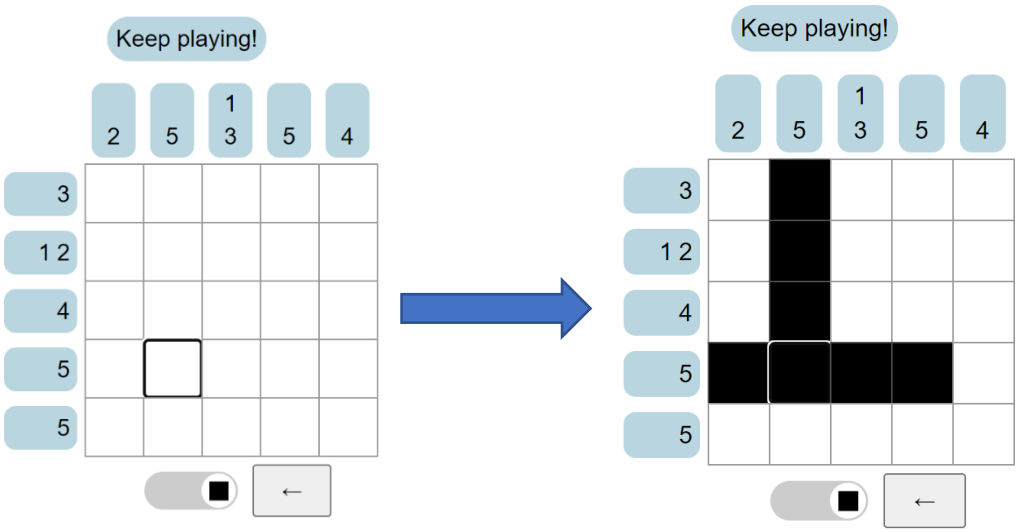
Reutilización de Predicados: Se destaca la reutilización de código, especialmente el predicado **selectRow/3**. Este predicado permite transformar una columna en una fila, lo que facilita la reutilización del predicado **checkClue** sin necesidad de desarrollar un algoritmo específico para esta tarea. Esto simplifica el diseño y mejora la modularidad del código.

Ejemplo de Uso del Predicado selectRow: Imaginemos que deseamos seleccionar la columna 2 del tablero al ingresar la grilla en el predicado selectRow. El arreglo resultante serían los siguientes:



Problemas

Durante el proceso de implementación en Prolog, se identificó un problema al utilizar **checkClue/3** se observó que las celdas sin instanciar se rellenaban automáticamente con el carácter "#" al intentar verificar si cumplían con sus pistas asociadas. Esto generaba resultados inesperados y afectaba la integridad del juego.



Para solucionar este problema, se reorganizaron los predicados que utilizaban las celdas pintadas, colocándolos al final. De esta manera, primero se verifica si las celdas no están pintadas antes de intentar realizar cualquier operación. Si la celda no está pintada, se procede con la verificación normal. Esta reorganización garantiza que las celdas no se llenen automáticamente y evita resultados inesperados.

Respecto a Game.js Inicialmente, se intentó calcular si el jugador había ganado después de cada movimiento utilizando una función. Sin embargo, se observó que este modulo funciona de manera asincrónica, lo que provocaba que a veces se llamara con instancias anteriores, lo que conducía a resultados incorrectos. Entonces se optó por utilizar useEffect en React para verificar si el jugador había ganado cada vez que una de las dos listas que representan el estado de satisfacción de las pistas se modificaba. Sin embargo, surgió un nuevo problema al definir el estado de estas listas como null, lo que provocaba que useEffect se llamara con null o undefined, aunque esto se verificara ocasionaba un fallo en ejecucion. Dado el desconocimiento en JavaScript, como solución temporal, se inicializa el estado de satisfacción como una lista vacía. Luego, en useEffect, se verifica que esta lista no esté vacía antes de llamar a la función checkWin. Aunque esta solución funciona, se reconoce que es poco elegante y podría mejorarse en el futuro para garantizar un código más limpio y legible.